

Public Key Handshake with CA-Based Server Authentication

1. Objective

Implement a simplified secure handshake protocol that demonstrates:

1. Ephemeral Diffie-Hellman key exchange
2. Session key derivation
3. Server authentication using an X.509 certificate issued by a Certificate Authority (CA)
4. Protection against Man-in-the-Middle (MITM) attacks

You are **not** implementing TLS.

You are implementing a minimal protocol that models the essential security concepts.

2. System Roles

You must implement three logical components:

- **CA (Certificate Authority)** — issues certificates
- **Server** — proves its identity and performs key exchange
- **Client** — verifies server identity and establishes secure session

All components may run as separate programs or as separate modules.

3. Constraints

- Language: Java or Kotlin
 - Allowed:
 - `BigInteger`
 - `SecureRandom`
 - Standard hashing (SHA-256)
 - Standard HMAC
 - Standard X.509 certificate parsing & signature verification APIs
 - Not allowed:
 - TLS libraries (SSLSocket, HTTPS client, etc.)
 - High-level crypto frameworks that perform handshake automatically
-

4. Part A — Certificate Authority (CA)

4.1 CA Responsibilities

The CA must:

1. Generate its own long-term key pair:
 - `caPrivateKey`
 - `caPublicKey`
2. Issue a server certificate containing:
 - Server identity (e.g., `"example.com"`)
 - Server public key
 - CA signature over certificate contents

4.2 Output

- `serverCertificate`
 - `caPublicKey` (to be trusted by Client)
-

5. Part B — Server Implementation

5.1 Server Startup

Server must load:

- `serverCertificate`
- `serverCertificatePrivateKey`
- DH parameters:
 - `primeModulus`
 - `generator`

5.2 On Receiving ClientHello

Server must:

1. Generate:
 - `serverNonce`
 - `serverEphemeralDhPrivate`
 - `serverEphemeralDhPublic`

2. Build handshake transcript:

```
transcript = clientNonce + serverNonce + clientEphemeralDhPublic +  
serverEphemeralDhPublic
```

3. Compute:

```
transcriptHash = SHA-256(transcript)  
serverHandshakeSignature = Sign(transcriptHash,
```

```
serverCertificatePrivateKey)
```

4. Send:

```
ServerHello {  
    serverNonce  
    serverEphemeralDhPublic  
    serverCertificate  
    serverHandshakeSignature  
}
```

5. Compute shared secret:

```
sharedSecret = modPow(clientEphemeralDhPublic, serverEphemeralDhPrivate, primeModulus)
```

6. Derive handshake key:

```
handshakeKey = KDF(sharedSecret, clientNonce, serverNonce)
```

6. Part C — Client Implementation

6.1 Client Startup

Client must load:

- `trustedCaPublicKey`

6.2 ClientHello

Client must:

1. Generate:

- `clientNonce`
- `clientEphemeralDhPrivate`
- `clientEphemeralDhPublic`

2. Send:

```
ClientHello {  
    clientNonce  
    clientEphemeralDhPublic  
}
```

6.3 On Receiving ServerHello

Client must:

Step 1 — Verify Certificate

- Validate that `serverCertificate` is signed by `trustedCaPublicKey`.
- Extract `serverCertificatePublicKey`.

If verification fails → Abort.

Step 2 — Verify Server Handshake Signature

Rebuild transcript:

```
transcript = clientNonce + serverNonce + clientEphemeralDhPublic +  
serverEphemeralDhPublic
```

Compute:

```
transcriptHash = SHA-256(transcript)
```

Verify:

```
Verify(serverHandshakeSignature, transcriptHash,  
serverCertificatePublicKey)
```

If verification fails → Abort.

Step 3 — Compute Shared Secret

```
sharedSecret = modPow(serverEphemeralDhPublic, clientEphemeralDhPrivate,  
primeModulus)
```

Step 4 — Derive Session Key

```
handshakeKey = KDF(sharedSecret, clientNonce, serverNonce)
```

Step 5 — Send ClientFinished

```
clientHandshakeMac = HMAC(handshakeKey, transcriptHash)  
  
Send ClientFinished {  
    clientHandshakeMac  
}
```

7. Server Verification of ClientFinished

Server must verify:

```
HMAC(handshakeKey, transcriptHash) == clientHandshakeMac
```

If valid → Secure session established.

8. Required Security Tests

Students must demonstrate:

1. Normal handshake success.
 2. Tampering with `serverEphemeralDhPublic` causes signature verification failure.
 3. Using a fake certificate (not signed by CA) causes certificate validation failure.
 4. Modifying transcript data causes MAC verification failure.
-

9. Deliverables

1. Source code (CA, Server, Client)
 2. README with execution instructions
 3. Report explaining:
 - How DH produces shared secret
 - How CA binds identity to public key
 - Why signing transcript prevents MITM
 - Why session key derivation is necessary
 4. Logs demonstrating all required security tests
-

10. Grading Criteria

Category	Description
Correct DH implementation	Shared secret equality
Proper certificate verification	CA validation logic
Correct signature verification	Transcript integrity protection
Secure session key derivation	KDF correctness
Attack simulation	Demonstrated failure cases
Documentation quality	Clear security reasoning

